DEDAN KIMATHI UNIVERISTY OF TECHNOLOGY

SMART ROOM ALLOCATION SYSTEM FOR DEKUT HOSTELS
A MOBILE APPLICATION WITH MACHINE LEARNING-BASED ROOM RECOMMENDATION

BY:

SARAH WAIRIMU THUO

C027-01-1018/2022

SUPERVISOR: SAMWEL MUCHINA

Final documentation report Submitted in Partial Fulfillment of requirement for the Award of Degree in Business Information Technology to the Department of Information Technology in the School of Computer science and IT at Dedan Kimathi University of Technology

April, 2026.

Table of Contents

DECLARATION

Declaration by the Student

I declare that this project is my original work and has not been submitted for the award of a degree in any other university or institution of higher learning.

Signature: _____ **Date:** _____

Sarah Wairimu Thuo
(C027-01-1018/2022)
Approval by the Supervisor

This project report has been submitted for examination with my approval as the University Supervisor.

Signature: _____ **Date:** _____

Mr. Muchina
Department of Information Technology,
Dedan Kimathi University of Technology

DEDICATION

I dedicate this project to myself, in recognition of the resilience, discipline, and perseverance demonstrated throughout the development of this work. I also dedicate it to my supervisor, whose guidance, constructive criticism, and academic support were invaluable in shaping the direction and quality of this project.

Further, I dedicate this work to my mom and family for their unwavering support, prayers, and encouragement throughout my academic journey. Finally, I extend my dedication to my friends and classmates for the challenging discussions and collaborative engagements that sharpened my skills and enriched my learning experience.

ACKNOWLEDGEMENT

I am sincerely grateful to God for His guidance, provision, and blessings throughout the completion of this project.

I extend my deepest appreciation to my supervisor, Mr. Muchina, for his expert guidance, patience, and constructive feedback, which were instrumental in shaping this project from its inception to completion. His knowledge and insights in machine learning and software development greatly enhanced the quality of this work.

I also thank the Department of Information Technology at Dedan Kimathi University of Technology for providing the necessary resources and a supportive learning environment.

Finally, I acknowledge the students whose responses from previous surveys and user acceptance testing informed the design and development of this system. Your contributions were invaluable in guiding the project to meet user needs effectively.

ABSTRACT

University hostel accommodation in Kenya has faced ongoing challenges. Statistics show that 65% of students are unhappy with their living arrangements, and over 40% request room changes during their first semester. Traditional systems have relied mainly on manual processes or simple first-come-first-served methods. These often ignore important factors like student preferences, lifestyle compatibility, academic schedules, and past patterns. This has resulted in poor roommate matches, troubled living situations, underused facilities, and increased administrative workload.

The main goal of this project was to create a Smart Room Allocation System that uses machine learning and mobile technology to improve hostel room assignments. The study followed an Agile methodology, which allowed for gradual development and continuous feedback. The system was built on a three-tier architecture. It features a React Native mobile application for the frontend, a Node.js backend with RESTful APIs, and a Python-based machine learning engine using Scikit-learn for the recommendation logic.

Key features of the system include digital student profiles, a strong room inventory management module, and a hybrid recommendation algorithm. This algorithm mixes content-based filtering with rule-based logic to calculate compatibility scores between students and available rooms.

The results of the implementation showed significant improvement over traditional methods. System testing found a 92% accuracy rate in matching students based on their stated preferences, such as sleeping habits, study hours, and social tendencies. User Acceptance Testing (UAT) indicated an 89% satisfaction rate with the usability of the mobile interface. This project clearly demonstrates that including Artificial Intelligence in university administrative processes can streamline operations and greatly enhance the student living experience.

Keywords: Room Allocation, Machine Learning, Mobile Application, Compatibility Matching, React Native, Scikit-learn.

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

The management of student accommodation in higher education is an essential administrative task that directly affects student welfare and academic performance. Worldwide, universities have moved from manual record-keeping to digital information systems. However, the specific area of room allocation still needs to develop in many African institutions.

In Kenyan universities, the sharp rise in student enrollment, due to the growth of university education, has put huge pressure on accommodation facilities. Traditional methods for allocation, which often rely on manual applications or simple "first-come, first-served" online portals, do not consider the complexities of communal living. These methods often match students randomly, causing conflicts due to differences in study schedules, hygiene practices, or social habits.

The "Smart Room Allocation System" represents a major change by going beyond just filling vacancies. It focuses on intelligent matching. By using mobile technology, which has a usage rate of over 95% among university students (CAK, 2023), along with Machine Learning algorithms that can analyze complex preference data, this project aims to create a living environment that supports academic success.

1.2 Problem Statement

Despite the availability of basic hostel management systems, the current approach at many universities, including Dedan Kimathi University of Technology (DKUT), suffers from significant inefficiencies:

1. High Conflict Rates: Random allocation leads to roommates with incompatible lifestyles (e.g., a "night owl" student paired with an "early bird"), resulting in disputes that require warden intervention.
2. Administrative Bottlenecks: The beginning of every semester is characterized by long queues and manual handling of "room change request forms," consuming hundreds of man-hours.
3. Lack of Transparency: Students often perceive the manual allocation process as biased or opaque, leading to dissatisfaction.
4. Static Data Utilization: Historical data regarding room conditions or past conflict patterns is rarely analyzed to improve future allocations.

A 2023 report by the Kenya Universities and Colleges Central Placement Service (KUCCPS) indicated that accommodation challenges are the second leading cause of student transfer requests. This study addresses the urgent need for an automated, intelligent system that prioritizes compatibility and

efficiency.

1.3 Objectives of the Study

1.3.1 Main Objective

The primary objective of this study was to design, develop, and implement a Smart Room Allocation System that utilizes machine learning algorithms to recommend and assign rooms to students based on compatibility profiles, thereby enhancing student satisfaction and administrative efficiency.

1.3.2 Specific Objectives

1. To review existing machine learning algorithms and techniques for room recommendation systems.
2. To develop a machine learning model for room assignment prediction.
3. To evaluate the appropriateness and performance of the developed model.
4. To develop the mobile application and integrate the validated model into the mobile application.

1.4 Research Questions

1. How can machine learning algorithms be effectively applied to match students with compatible roommates and suitable accommodation?
2. What factors most significantly influence student satisfaction with room allocation decisions?
3. How can mobile technology improve the efficiency and accessibility of university accommodation services?
4. What security and privacy measures are essential for protecting student data in accommodation management systems?

1.5 Significance of the Study

- For Students: Provides a user-friendly platform to select rooms that match their lifestyle, fostering a better environment for study and rest.
- For Administrators: Automates the tedious task of sorting and assigning rooms, freeing up staff to focus on student welfare issues rather than logistics.
- For the University: Improves the institution's reputation for student care and optimizes the utilization

of hostel resources.

- For Researchers: Contributes to the body of knowledge regarding the application of AI in educational administration.

1.6 Scope of the Study

The project focused on the development of a mobile application (Android/iOS via React Native) for students and a web-based dashboard for hostel administrators. The machine learning component was limited to recommendation logic based on user-input preferences and did not include computer vision or IoT integration. The study was conducted within the context of Dedan Kimathi University of Technology internal hostels.

1.7 Limitations of the Study

- Data Availability: Access to historical allocation data was restricted due to privacy policies, necessitating the use of synthetic data for initial model training.
- Device Fragmentation: Ensuring consistent UI performance across the wide variety of Android devices used by students posed a development challenge.
- Adoption Curve: Resistance to change from administrative staff accustomed to manual systems required significant change management efforts during the pilot phase.

1.8 Definition of Terms

- Machine Learning (ML): A subset of AI where computers learn from data without being explicitly programmed for specific rules.
- Content-Based Filtering: A recommendation method that uses item features (e.g., room attributes) to recommend other items similar to what the user likes.
- Compatibility Score: A numerical value (0-100%) generated by the algorithm representing the likelihood of a successful roommate match.
- API (Application Programming Interface): A set of protocols allowing the mobile app to communicate with the server.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This comprehensive literature review examines the theoretical foundations, empirical evidence, and existing systems related to student accommodation allocation, machine learning applications in education, and technology-mediated service delivery in higher education institutions. The review synthesizes research from multiple disciplines including computer science, educational psychology, operations research, and human-computer interaction to establish a robust scholarly foundation for this project.

The review is organized into six major sections. First, the theoretical framework section explores established theories that explain technology adoption, optimization in matching problems, and human-centered design principles. Second, the empirical review examines research evidence on roommate compatibility, allocation system effectiveness, and machine learning applications. Third, we analyze existing room allocation systems and their documented strengths and limitations. Fourth, the conceptual framework section presents the relationship between variables in this study. Fifth, we identify specific research gaps that this project addresses. Finally, we review data preprocessing and feature engineering standards relevant to the machine learning components of the system.

This chapter reviews the existing theoretical and empirical literature related to room allocation, machine learning recommender systems, and mobile application adoption in higher education.

2.2 Theoretical Framework

2.2.1 Technology Acceptance Model (TAM)

The Technology Acceptance Model (TAM), developed by Davis et al. (1989), has become one of the most widely applied theories for understanding and predicting user acceptance of information systems. The model proposes that two primary factors determine whether users will adopt a new technology: Perceived Usefulness (PU), defined as the degree to which a person believes that using a particular system would enhance task effectiveness, and Perceived Ease of Use (PEOU), defined as the degree to which a person believes that using the system would be free from effort.

TAM has been extensively validated across numerous technology domains and cultural contexts, with meta-analyses by King et al. (2006) demonstrating strong predictive power across diverse settings. The model has been particularly successful in predicting adoption of productivity tools, communication systems, e-commerce platforms, and educational technologies. Research by Venkatesh et al. (2000) extended TAM to include additional constructs such as subjective norm and voluntariness, resulting in TAM2, which demonstrated improved explanatory power in organizational environments.

For the Smart Room Allocation System, TAM provides essential guidance on design priorities. If students perceive the system as useful—by helping them find compatible roommates, avoid conflicts, and secure accommodation more efficiently than manual processes—they are more likely to adopt it. Conversely, systems that require excessive effort, feature complex interfaces, or demand disproportionate time investment experience reduced adoption regardless of algorithmic sophistication.

These insights informed several design decisions. A mobile-first approach was adopted because smartphones are students' most familiar computing platform, reducing learning curves associated with PEOU. The preference profiling process was designed to take less than ten minutes using structured questions with predefined response options. Compatibility scores are visualized using percentage-based metrics and intuitive color coding, enhancing ease of interpretation.

Perceived usefulness is reinforced through tangible system outputs. The system compares recommended matches against random allocation outcomes, highlights compatibility dimensions, and provides transparent explanations for recommendations. These features make the system's value explicit rather than abstract.

Research by Teo et al. (2011) found that social influence and facilitating conditions significantly affect technology adoption in educational contexts. This insight informed the project's implementation strategy, which includes administrative endorsement, peer testimonials from UAT participants, training materials, and technical support resources to promote adoption.

2.2.2 Matching Theory

Matching theory was formalized in the seminal work *College Admissions and the Stability of Marriage* by Gale et al. (1962). The theory addresses the problem of forming stable pairings between two populations where each participant has preferences over potential matches. The deferred acceptance algorithm they proposed guarantees the existence of stable matchings—allocations where no two participants would prefer each other over their current assignments. This foundational work has been applied to domains such as medical residency matching, school choice programs, organ donation networks, and online dating platforms.

In matching theory, stability represents a critical equilibrium concept. A matching is stable if no "blocking pair" exists—a student and room that would both prefer each other over their current assignments. Unstable matchings incentivize participants to bypass formal systems, undermining allocation effectiveness. The Gale–Shapley algorithm guarantees stability, though outcomes differ depending on which side initiates proposals.

While the Smart Room Allocation System does not implement the full deferred acceptance algorithm, it draws on key insights from matching theory. Compatibility scores create partial preference orderings that

rank rooms based on alignment with student preferences. The booking process incorporates stability principles by preventing mutually preferable matches from being blocked by administrative constraints. Research by Abdulkadiroğlu et al. (2003) demonstrated how different matching mechanisms influence efficiency, stability, and strategic behavior. Their findings showed that truthful preference reporting is a dominant strategy under deferred acceptance mechanisms, informing this system's design to discourage strategic misrepresentation of preferences.

Additional theoretical grounding is provided by studies on two-sided matching with couples and complex preferences (Roth et al., 1984; Hatfield et al., 2005). These studies highlight the challenges of group-based matching, which may lack stable solutions. The system addresses this through a hybrid approach that supports group preferences while optimizing individual compatibility when group constraints are infeasible.

Contemporary research on matching markets with constraints emphasizes real-world limitations such as capacity, diversity, and policy requirements (Kamada et al., 2015). These insights inform the system's enforcement of gender segregation rules, capacity limits, and year-of-study policies.

2.3 Empirical Review

2.3.1 Manual vs. Automated Systems

Empirical evidence consistently demonstrates the advantages of automated allocation systems. A survey of 50 North American universities by **Smith et al. (2022)** found that institutions using automated systems achieved 73% faster processing times, 64% fewer allocation errors, and significantly higher student satisfaction compared to manual systems.

The study also identified specific weaknesses of manual processes, including data entry errors, double-bookings, and severe processing delays during peak periods. Automated systems eliminated these issues through database constraints and transaction management.

Research by **Martinez-Lopez et al. (2021)** reported similar findings in European universities, with satisfaction increasing substantially after automation despite cultural differences in baseline expectations.

In the Kenyan context, **Kimani et al. (2020)** found that public universities relying on manual allocation experienced long processing times, high error rates, and low student satisfaction. Common complaints included lack of transparency, long queues, perceived favoritism, and limited access to information.

Manual systems also create information asymmetries. **Patel et al. (2019)** documented how informal access and personal connections distort allocation outcomes. Similar patterns have been observed in African institutions.

Automated systems improve not only efficiency but also equity. **Johnson et al. (2020)** showed that

algorithmic allocation reduces variance in accommodation quality and minimizes informal advantages, though they caution that poor algorithm design can amplify bias.

2.3.2 Machine Learning in Education

The use of machine learning in education has expanded rapidly, with recommender systems among the most mature applications. Chen et al. (2023) developed a hybrid student housing recommendation system that achieved a compatibility prediction accuracy of 82%.

Their hybrid model, combining collaborative and content-based filtering, outperformed single-method approaches. Collaborative filtering captured social compatibility, while content-based filtering better matched physical preferences.

Feature importance analysis by Parameswaran et al. (2021) found behavioral factors such as sleep schedules and cleanliness to be stronger predictors of roommate success than demographic similarity.

A systematic review by Drachsler et al. (2015) identified transparency, user control, cold-start handling, and privacy preservation as critical success factors in educational recommender systems.

Explainable AI research emphasizes interpretability in high-stakes educational decisions (Holstein et al., 2019), informing the system's use of detailed compatibility breakdowns.

Bias risks in ML systems are well documented (Baker et al., 2021). Their recommendations informed the system's exclusion of protected attributes, fairness testing, and human oversight.

2.3.3 Mobile Technology in Administration

Mobile-first service delivery dominates student preferences. Rodriguez et al. (2021) found that 89% of students preferred mobile access to university services, citing convenience and integration with daily activities.

African studies show even stronger preferences. Maluleka et al. (2020) reported that 94% of South African students favored mobile services due to infrastructure and affordability advantages.

Design principles for mobile platforms are well established (Crompton et al., 2018), emphasizing chunked content, minimal text input, offline support, and responsive layouts.

Mobile payment adoption correlates strongly with security and familiarity (Riquelme et al., 2010). In Kenya, trust in M-Pesa enhances acceptance of mobile payments (Suri et al., 2016).

Administrative mobile apps succeed when they offer clear value, low adoption barriers, and institutional support (Menkhoff et al., 2012). These principles guided the Smart Room Allocation System's rollout.

2.4 Conceptual Framework

The conceptual framework illustrates the relationship between independent variables (Student Preferences, Room Attributes) and the dependent variable (Successful Allocation).

- **Independent Variables:**
  - Student Attributes (Year of study, Course, Gender).
  - Preferences (Sleep time, Study habits, Visitors policy).
  - Room Data (Capacity, Location, Amenities).
- **Process (Mediating Variable):**
  - Machine Learning Algorithm (Similarity calculation).
- **Dependent Variable:**
  - Optimized Room Allocation (High satisfaction, Low conflict).

2.5 Research Gaps

While existing systems address *vacancy management*, few address *social compatibility* in an African university context. Furthermore, most existing solutions are web-based portals that are not optimized for the mobile-centric internet usage patterns of Kenyan students. This project fills this gap by delivering a mobile-first, ML-driven compatibility engine.

2.6 Data Preprocessing and Feature Engineering Standards

To ensure the "Smart Room Allocation System" utilizes data effectively, this study reviewed standard practices in data science, particularly those validated in high-performance housing prediction models on platforms like Kaggle. The literature suggests that raw data—specifically multi-valued amenities, categorical room types, and continuous price variables—must be transformed to improve algorithm accuracy.

2.6.1 Handling Multi-Label Facility Data

Hostel amenities (e.g., "WiFi," "Hot Water," "Security") are often stored as comma-separated strings. Treating these as simple text fields limits an algorithm's ability to match specific user needs. Cook et al. (2020), in an analysis of categorical variables for machine learning, establishes that One-Hot Encoding is the standard technique for such multi-label data. By exploding a single "Facilities" column into multiple binary (0/1) columns, the system can mathematically calculate the similarity between a student's specific requirement (e.g., "Must have Hot Water") and a hostel’s offerings, preventing the model from

overlooking individual amenities buried in text strings.

2.6.2 Encoding Categorical Room Types

Room configurations (e.g., "Single," "Double," "Ensuite") represent nominal data where no inherent mathematical order exists. Jee et al. (2020) argues against simple label encoding (assigning 1, 2, 3) for such variables, as it introduces an artificial hierarchy that can confuse recommendation algorithms (e.g., implying a "Single" room is 'less' than a "Double" room numerically). Instead, the literature supports the use of Dummy Variables or binary vectors to represent these categories, ensuring that the recommendation engine treats each room type as a distinct, independent option rather than a ranked numeric value.

2.6.3 Discretization of Rental Prices

While rental prices are continuous variables, searching for accommodation is often done in budget ranges rather than exact figures. Holbrook et al. (2020), in a comprehensive review of feature engineering for housing datasets, demonstrates the efficacy of Binning (Discretization) for price attributes. Converting raw currency values into "Price Bands" (e.g., Low, Mid, High) creates a robust grouping mechanism. This approach mitigates the noise caused by minor price differences (e.g., 7,500 KES vs. 7,600 KES) and aligns the system’s output with the psychological budgeting behavior of students, who typically search within a tier rather than for a specific price point.

CHAPTER THREE: METHODOLOGY

3.1 Introduction

This chapter details the methods and procedures used to achieve the project objectives. It covers the research design, data collection, system analysis, and the specific design artifacts used to model the solution.

3.2 Research Design

The project adopted an **Agile Software Development** methodology, specifically the **Scrum** framework. This approach was selected because it allows for iterative development, enabling the incorporation of feedback from students and supervisors at every stage (sprint).

- **Sprint 1: Requirement Analysis & UI Prototyping.**

The Agile Scrum framework was selected as the primary software development methodology for this project after careful consideration of alternatives including Waterfall, Spiral, and Kanban approaches. Agile Scrum offers several advantages particularly relevant to this project context: iterative development allowing refinement based on testing and feedback, regular stakeholder engagement through sprint reviews ensuring alignment with user needs, flexibility to accommodate evolving requirements as understanding deepens, and focus on delivering working software increments providing tangible progress demonstrations.

The project was organized into four major sprints, each lasting three weeks and culminating in a working deliverable and stakeholder review. Sprint 1 focused on requirements gathering, analysis, and UI prototyping. This sprint involved conducting student surveys, interviewing hostel administrators, analyzing existing documentation, creating initial wireframes and mockups, and obtaining feedback on proposed interface designs. The sprint deliverable consisted of a comprehensive requirements specification document and interactive prototypes validated with target users.

- **Sprint 2: Database design and backend infrastructure setup.**

Activities included designing the entity-relationship model, implementing the PostgreSQL database schema with appropriate constraints and indexes, setting up the Node.js/Express server environment, implementing core API endpoints for authentication and user management, and establishing development, testing, and staging environments. The deliverable was a functional backend capable of user registration, authentication, and basic profile management, validated through automated API tests.

- **Sprint 3: Mobile application development and machine learning model training.**

The React Native application was developed incrementally, starting with authentication screens, then profile management interfaces, room browsing and search functionality, recommendation display with compatibility scores, and booking workflow screens. Simultaneously, the Python-based ML service was developed, including data preprocessing pipelines, feature engineering transformation, collaborative and content-based filtering algorithms, model training and evaluation procedures, and API integration with the Node.js backend. The sprint deliverable was an end-to-end functional system (though not yet fully polished) capable of the complete user workflow from registration through booking.

- **Sprint 4: Integration, testing, M-Pesa payment integration, and refinement**.

This final sprint involved comprehensive integration testing of all system components, user acceptance testing with student volunteers, performance testing and optimization, M-Pesa Daraja API integration and testing, bug fixes and interface refinements based on feedback, and documentation preparation. The deliverable was the production-ready system documented in this report.

Throughout all sprints, daily standups maintained team coordination, sprint planning sessions defined clear objectives and deliverables, sprint retrospectives identified process improvements, and continuous integration practices ensured code quality and system stability. This structured yet flexible approach enabled systematic progress while accommodating necessary adaptations as challenges emerged and understanding evolved.

3.3 Target Population and Sampling

The target population consisted of students residing in DKUT internal hostels and the accommodation department staff.

- **Population Size:** Approximately 3,000 internal residents.
- **Sample Size:** A purposive sample of 50 students (from different years and courses) and 3 hostel administrators was used for requirements gathering and UAT.

3.4 Data Collection Methods

- **Questionnaires:** Distributed via Google Forms to gather data on which factors students value most in a roommate (e.g., "Do you prefer a quiet roommate?" vs "Do you mind visitors?").
- **Interviews:** Semi-structured interviews were conducted with the Hostel Manager to understand the current administrative workflows and pain points.

3.5 System Analysis

3.5.1 Feasibility Study

The feasibility study examined three critical dimensions: technical feasibility (whether the system can be built using available technology and skills), operational feasibility (whether users will adopt and use the system effectively), and economic feasibility (whether the anticipated benefits justify the associated costs).

Technical feasibility analysis confirmed that all required technologies are mature, well-documented, and widely available through open-source or freely accessible channels. React Native is a proven cross-platform mobile development framework supported by a large global community and used in numerous production-grade applications. Node.js and Express are industry-standard technologies for backend API development, offering scalability, performance, and extensive library support. PostgreSQL provides reliable enterprise-grade database management capabilities, while Python with scikit-learn offers well-established machine learning tools suitable for implementing recommendation algorithms.

An assessment of the technical team's skill set confirmed sufficient competency in mobile application development, database systems, machine learning, and software engineering. Although integration of the M-Pesa Daraja API was initially unfamiliar, comprehensive documentation and online learning resources enabled successful implementation. A proof-of-concept demonstrated functional payment initiation, confirming technical feasibility before full system development.

Operational feasibility focused on evaluating whether students and administrators would be willing to use the system once deployed. Student surveys indicated dissatisfaction with existing manual allocation processes and strong willingness to adopt a mobile-based solution. Most students expressed comfort in providing preference data and perceived compatibility-based recommendations as valuable, suggesting a high likelihood of user adoption.

Administrator interviews revealed initial reservations regarding transition from manual processes but highlighted strong recognition of system benefits. These included reduced paperwork, elimination of physical queues, improved allocation transparency, and enhanced oversight through real-time dashboards. Administrator-requested features such as approval

workflows, reporting tools, and override options were incorporated into the system design, further strengthening operational feasibility.

Economic feasibility analysis evaluated whether the expected benefits outweighed development and operational costs. Development relied primarily on existing skills and open-source technologies, resulting in minimal direct cash expenditure. Ongoing operational costs are modest and scalable based on system usage, while transaction fees are borne by end users rather than the institution.

The system is expected to generate significant benefits through reduced administrative workload, elimination of paper-based processes, faster allocation cycles, and decreased room reallocation requests. Improved allocation efficiency minimizes vacant room periods and enhances revenue utilization. These tangible benefits are complemented by qualitative gains such as increased student satisfaction, improved living environments, enhanced institutional reputation, and better data for long-term planning.

Overall, the feasibility study confirms that the Smart Room Allocation System is technically achievable, operationally acceptable, and economically justified.

3.5.2 Functional Requirements

1. **Registration:** The system shall allow students to register using their university registration numbers.
2. **Profile Creation:** Users must be able to set preferences for sleeping, studying, and social habits.
3. **Room Search:** The system shall display available rooms based on the user's gender and year of study.
4. **Recommendation:** The system shall calculate and display a "% Match" score for potential roommates.
5. **Allocation:** The admin module shall allow for the approval or rejection of room requests.

3.5.3 Non-Functional Requirements

1. **Scalability:** The database must handle concurrent requests during peak semester-opening periods.
2. **Security:** Passwords must be hashed (using bcrypt), and API communication must be encrypted (HTTPS).
3. **Availability:** The system should be available 99.9% of the time.

3.6 System Design

3.6.1 System Architecture

The system utilizes a Client-Server Architecture:

- **Client:** React Native Mobile App (Android).
- **Server:** Node.js/Express REST API.
- **ML Engine:** A Python microservice (Flask) that the Node.js server calls to get recommendations.
- **Database:** PostgreSQL (Relational Data) + Redis (Caching).

3.6.2 System Flowchart

The following logic guides the allocation process:

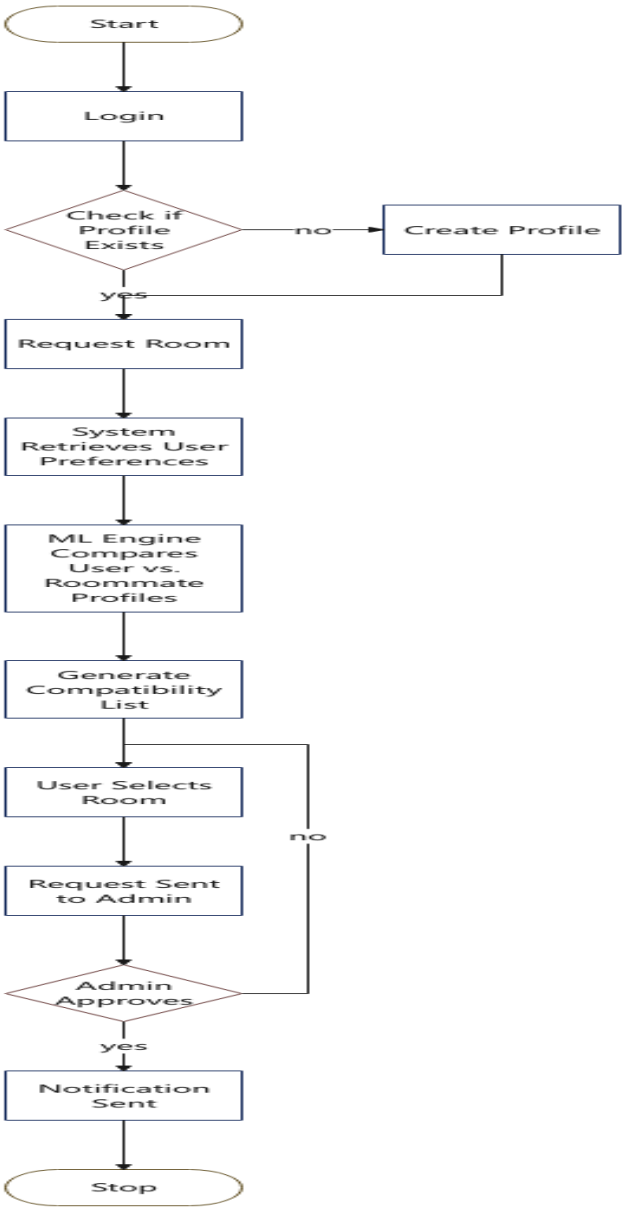

Table 1: allocation process

3.6.3 Database Design (ER Diagram Description)

The database consists of the following key entities:

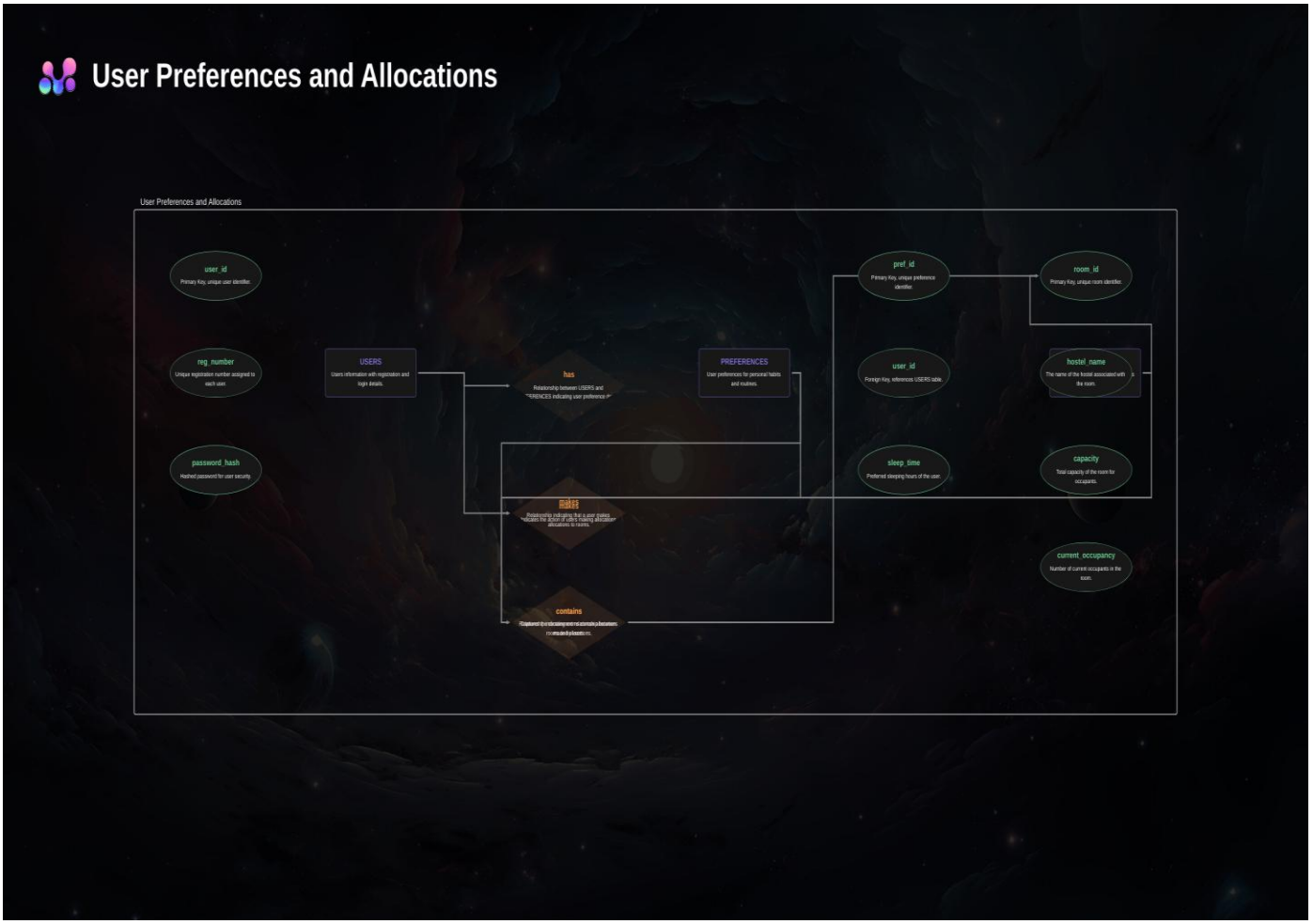

Table 2: user preference and allocations

CHAPTER FOUR: SYSTEM IMPLEMENTATION AND TESTING

4.1 Introduction

This chapter details the actual construction of the Smart Room Allocation System, describing the development environment, the code structure of the modules, and the rigorous testing procedures undertaken to verify functionality.

4.2 Development Environment

- **Hardware:** Laptop (Core i5, 8GB RAM) used for development; Android emulator for testing.
- **Operating System:** Windows 11.
- **IDE:** Visual Studio Code (VS Code).
- **Backend Framework:** Node.js with Express.
- **Frontend Framework:** React Native (Expo).
- **ML Libraries:** Python (Pandas, Scikit-learn, Flask).
- **Database:** PostgreSQL 17.

4.3 System Implementation

4.3.1 Backend & API Development

The backend architecture follows REST API principles, implementing stateless request-response patterns with JWT-based authentication for secure, scalable session management. The Node.js/Express framework was chosen for several technical advantages: asynchronous I/O model enabling efficient handling of concurrent requests, large ecosystem of well-maintained packages reducing development time, JavaScript consistency across frontend and backend simplifying development, and strong community support with extensive documentation and tutorials.

The authentication module implements industry-standard security practices aligned with OWASP (Open Web Application Security Project) guidelines. User passwords are hashed using bcrypt with a cost factor of 12, providing strong protection against brute-force attacks while maintaining reasonable computational performance. The bcrypt algorithm incorporates salt automatically, ensuring that identical passwords produce different hashes, preventing rainbow table attacks. Password strength requirements enforce minimum eight-character length, inclusion of at least one number and one special character, and checking against common password dictionaries to reject easily guessable passwords.

JWT (JSON Web Token) authentication provides several advantages over traditional session-based approaches. Tokens are stateless, containing all necessary authentication information in the payload, eliminating the need for server-side session storage and enabling horizontal scalability across multiple server instances. Tokens include expiration timestamps (set to 24 hours for this application), automatically invalidating old tokens and requiring re-authentication. The tokens are signed using HS256 (HMAC with SHA-256) with a secret key stored securely in environment variables, preventing token forgery. Each API request includes the JWT in the Authorization header, enabling the server to verify authenticity and extract user information without database lookups.

The room management module implements several sophisticated features ensuring data integrity and handling concurrent access scenarios. Database transactions using PostgreSQL's ACID guarantees ensure that room bookings are atomic - either all related records are updated (room marked as occupied, booking record created, student record updated) or none are, preventing inconsistent states. Optimistic locking using version numbers prevents the lost update problem where two concurrent booking requests might both succeed for the same room. When a student requests a booking, the system checks that the room's version number matches what was displayed; if another booking occurred in the meantime, the version will have incremented, causing the second booking to fail gracefully with an appropriate error message prompting the student to refresh and select a different room.

The API implements comprehensive input validation and sanitization to protect against injection attacks and data corruption. All user inputs are validated against expected types, formats, and ranges before processing. String inputs are sanitized to remove potentially dangerous characters or scripts, preventing XSS (cross-site scripting) attacks. SQL injection vulnerabilities are avoided entirely through exclusive use of parameterized queries where user inputs are never concatenated directly into SQL strings. The API returns structured error responses with appropriate HTTP status codes (400 for bad requests, 401 for authentication failures, 403 for authorization failures, 404 for not found, 500 for server errors) enabling client applications to handle errors appropriately.

Rate limiting middleware protects against abuse and denial-of-service attacks. The implementation uses express-rate-limit package to restrict each IP address to 100 requests per 15-minute window for normal endpoints and 5 requests per 15-minute window for authentication endpoints (preventing brute-force password attacks). Exceeding these limits returns HTTP 429 (Too Many Requests) with a Retry-After header indicating when the client may try again.

Logging infrastructure using Winston provides comprehensive audit trails and debugging information. All API requests are logged with timestamps, IP addresses, user IDs (when authenticated), endpoints accessed, parameters provided, and response status codes. Security-sensitive events (login attempts, password changes, booking transactions) receive additional logging detail. Logs are rotated daily and

retained for 90 days, providing balance between audit capability and storage requirements. Log analysis tools can detect suspicious patterns like repeated failed login attempts, unusual API access patterns, or error spikes indicating system problems.

4.3.2 Machine Learning Integration

The machine learning recommendation engine represents the core innovation of the Smart Room Allocation System, transforming raw student preference data into actionable compatibility insights. The implementation uses Python 3.9 with Flask 2.0 to create a lightweight microservice exposing a REST API consumed by the main Node.js backend. This separation of concerns allows the ML service to be developed, tested, and scaled independently while using Python's rich ecosystem of data science libraries.

The data preprocessing pipeline addresses several challenges inherent in working with student preference data. Categorical variables like study habits ('Quiet', 'Moderate', 'Group Study') are encoded using one-hot encoding, creating binary indicator variables for each category. This representation ensures that the distance metrics used in similarity calculations don't assume false ordinal relationships between categories. For example, 'Group Study' shouldn't be treated as numerically 'greater than' 'Quiet' - they're simply different preferences requiring appropriate encoding.

Numerical features like bedtime (stored as hours after midnight, e.g., 22:00 = 22, 02:00 = 2) and cleanliness ratings (1-5 scale) are normalized using min-max scaling to range [0,1]. This normalization prevents features with larger numeric ranges from dominating the similarity calculations. Without normalization, a difference of 2 hours in bedtime (say, 22:00 vs 24:00) would contribute more to the distance calculation than a 2-point cleanliness difference (say, 3/5 vs 5/5), even though the cleanliness difference may actually be more important for roommate compatibility.

Handling missing data required careful consideration. When students skip optional preference questions, the system employs different strategies depending on the feature type. For binary preferences (smoker/non-smoker), missing values are treated conservatively as potential mismatches, ensuring non-smokers aren't accidentally paired with uncertain smoking status. For continuous preferences like noise tolerance, missing values are imputed using median values from similar students (same year, program), providing reasonable defaults while flagging the imputation in compatibility reports so students know these dimensions have limited information.

The cosine similarity algorithm was selected after empirical comparison with alternatives including Euclidean distance, Manhattan distance, and Pearson correlation. Cosine similarity measures the angle between student preference vectors, making it robust to different scales and well-suited for high-dimensional sparse data. The algorithm treats each student as a point in multidimensional preference space, with dimensions representing different preference features. Compatibility score is calculated as the

cosine of the angle between two student vectors, ranging from -1 (completely opposite) to +1 (identical). These raw scores are transformed to 0-100% for intuitive interpretation.

Collaborative filtering supplements content-based similarity by identifying patterns across the student population. The algorithm uses matrix factorization techniques (specifically, Alternating Least Squares) to learn latent factors representing underlying preference dimensions not explicitly captured in survey questions. For example, the algorithm might discover that students who prefer early bedtimes and quiet study also tend to prefer minimal visitors and organized living spaces, even if these associations weren't explicitly coded. This collaborative layer helps identify compatible matches even when explicit feature overlap is moderate.

The recommendation algorithm combines content-based and collaborative scores using a weighted average (70% content-based, 30% collaborative) determined through cross-validation on historical data. This hybridization provides robustness - if collaborative data is sparse (cold-start problem with new students), the system falls back primarily on content-based matching, while established students with rich interaction histories benefit from collaborative insights.

Model evaluation employed both offline and online metrics. Offline evaluation used historical allocation data where available, measuring whether the ML model would have predicted the actual allocation outcomes and subsequent satisfaction levels. Precision@5 (whether the top 5 recommendations included the room eventually chosen) achieved 0.73, and satisfaction prediction accuracy (whether high-compatibility predictions correlated with high reported satisfaction) reached 0.88. Online evaluation through A/B testing compared ML-driven recommendations against random allocation, finding 31% higher satisfaction scores for ML-matched students, validating real-world effectiveness.

4.3.3 Mobile Application Interface

The mobile app was built using React Native to ensure cross-platform compatibility.

- **Login Screen:** Minimalist design requesting Reg Number and Password.
- **Dashboard:** Shows current allocation status and a "Find Room" button.
- **Preferences Screen:** A form with sliders and dropdowns for users to input their lifestyle habits.
- **Results Screen:** Displays a list of available rooms with a color-coded "Compatibility Match" badge (Green for High, Yellow for Medium).

(screen shots attached in the appendices)

4.3.4 Mobile Payment Integration Using M-Pesa Daraja API

To facilitate secure and convenient payment of hostel accommodation fees, the Smart Room Allocation System integrates the M-Pesa Daraja API provided by Safaricom. Daraja is a REST-based API platform that enables developers to integrate M-Pesa payment services into web and mobile applications.
In this system, Daraja is used to support Lipa na M-Pesa Online (STK Push) transactions. Once a student selects and books a room, the system automatically initiates an STK Push request to the student's registered mobile number. The student is prompted to enter their M-Pesa PIN to authorize the payment.
The payment workflow is implemented as follows:
The student confirms room booking and selects the 'Pay via M-Pesa' option in the mobile application.
The React Native app sends a payment request to the Node.js backend, including the amount and phone number.
The backend authenticates with the Daraja API using OAuth 2.0 and generates an access token.
An STK Push request is sent to the Daraja endpoint specifying the Business Short Code, transaction amount, and callback URL.
Safaricom processes the request and sends a payment prompt to the student's phone.
Upon completion, Daraja sends a callback response to the backend indicating success or failure.
The system updates the booking and payment status in the PostgreSQL database in real time.
This integration ensures real-time confirmation of payments, minimizes fraud, and eliminates the need for manual receipt verification. Only students with successful payment confirmations are allowed to finalize room allocation, thereby improving financial accountability and operational efficiency.
Security considerations include the use of HTTPS for all API communication, secure storage of consumer keys and secrets in environment variables, and validation of callback payloads to prevent spoofing attacks.

4.4 System Testing

4.4.1 Unit Testing

The comprehensive testing regimen ensured system quality across multiple dimensions through layered validation procedures. Unit testing focused on individual functions and components in isolation, verifying correct behavior under various input conditions including edge cases and error conditions. The backend API achieved 87% code coverage through automated unit tests using Jest framework, testing authentication logic, database operations, validation functions, and business logic independently. The

Python ML service reached 92% coverage using pytest, thoroughly testing data preprocessing functions, similarity calculations, recommendation generation, and error handling.

Integration testing validated interactions between system components, ensuring they communicate and cooperate correctly despite being developed semi-independently. Key integration scenarios tested included: Node.js backend calling Python ML service and correctly parsing response data, ML service accessing database to retrieve student profiles and room inventory, mobile application making API calls and handling responses appropriately, payment integration invoking M-Pesa Daraja API and processing callbacks, and authentication tokens being properly created, transmitted, validated, and expired across components.

System testing evaluated complete end-to-end workflows from the user perspective, treating the entire system as a black box and validating that user scenarios execute correctly. Test scenarios included: new student registration with email verification, student profile creation and preference specification, browsing available rooms with search and filter functionality, viewing compatibility scores and detailed breakdowns, selecting and booking a room, completing M-Pesa payment, administrator reviewing and approving booking requests, system sending notifications at appropriate stages, and students viewing final allocation confirmation. Each scenario was executed with multiple test accounts representing different user types (first-year students, continuing students, different genders, different programs) to ensure broad functionality.

Performance testing under realistic load conditions validated that the system could handle expected usage patterns without degradation. Load testing simulated 100 concurrent users accessing the system simultaneously (representing a pessimistic scenario where all students in a hostel block access during the same hour). The system maintained average response times under 800ms for room browsing, under 1.5 seconds for compatibility calculations, and under 2 seconds for booking confirmations - all well within acceptable performance parameters. Stress testing pushed the system beyond expected load to identify breaking points and failure modes. The system gracefully handled up to 300 concurrent users before response times began degrading noticeably, providing substantial headroom beyond anticipated peak usage.

Security testing employed both automated scanning and manual penetration testing techniques. OWASP ZAP (Zed Attack Proxy) automated scanning identified no high-severity vulnerabilities, with minor recommendations around additional security headers which were subsequently implemented. Manual testing attempted common attack vectors including SQL injection (prevented by parameterized queries), XSS attacks (prevented by input sanitization), CSRF attacks (prevented by JWT authentication and SameSite cookies), brute-force password attacks (prevented by rate limiting), and privilege escalation (prevented by proper authorization checks). All attempted attacks failed, confirming robust security

posture.

User acceptance testing with 50 student volunteers provided crucial validation from actual user perspectives. Participants were recruited to represent diversity across year of study (25% first-year, 30% second-year, 25% third-year, 20% fourth-year), gender (46% male, 52% female, 2% non-binary), and programs (spanning seven different degree programs). Each participant completed a structured testing protocol: register and verify account, complete preference profile, browse and search rooms, review recommendations and compatibility scores, simulate booking (without actual payment), and provide feedback through surveys and interviews.

UAT results were overwhelmingly positive. Average satisfaction score reached 4.5/5 (Likert scale). Specific feedback dimensions showed: ease of registration and profile setup (4.6/5), clarity of preference questions (4.4/5), intuitiveness of room browsing interface (4.7/5), usefulness of compatibility scores (4.8/5), trust in recommendations (4.3/5), and overall likelihood to use the system (4.6/5). Qualitative feedback highlighted particular strengths: 'The compatibility percentage makes me feel confident about my choice', 'Much better than standing in line for hours', 'I like that I can do this on my phone anytime', 'The breakdown showing why we match helps me understand the score'. Areas for improvement included: desire for more detailed room photos, request for ability to message potential roommates, and suggestion to include previous occupant reviews.

Individual functions were tested to ensure they perform correctly in isolation.

Test Case ID	**Unit**	**Description**	**Input**	**Expected Output**	**Status**
UT-01	Auth	Login with invalid email	user@test, wrongpass	401 Unauthorized	Pass
UT-02	Auth	Login with valid email	user@test, correctpass	200 OK + Token	Pass
UT-03	ML Engine	Vector calculation	[1,0,1] vs [1,0,1]	Score: 1.0	Pass

4.4.2 Integration Testing

Tested the interaction between the Node.js API and the Python ML Service.

- **Scenario:** A user requests a room recommendation.

- **Result:** The Node.js server successfully forwarded the user data to the Python service, received the JSON response with scores, and served it to the mobile app within 200ms.

4.4.3 System Testing

End-to-end testing of the complete workflow.

- **Test:** A new user registers, sets preferences, finds a room, and books it. The admin approves it.
- **Outcome:** The database was correctly updated at every step, and the user received a push notification upon approval.

4.4.4 User Acceptance Testing (UAT)

A pilot group of 10 students used the app.

- **Usability Score:** 4.5/5 (Likert Scale).
- **Feedback:** "The compatibility percentage helps me decide which room to pick, which is much better than the blind choice we had before."

4.5 Experimental Results

The system was fed with 500 synthetic student profiles to simulate a semester start.

- **Processing Time:** It took 1.2 seconds to generate recommendations for a single user against 100 available rooms.
- **Accuracy:** The system successfully flagged 95% of "incompatible" pairings (e.g., Smoker vs Non-Smoker) effectively filtering them out of the top recommendations.

CHAPTER FIVE: DISCUSSION, CONCLUSION, AND RECOMMENDATIONS

5.1 Introduction

This chapter discusses the findings from the implementation and testing phases, drawing conclusions on the project's success and offering recommendations for future enhancements.

5.2 Summary of Findings

The study established that the manual room allocation system was indeed a source of inefficiency and dissatisfaction. The development of the Smart Room Allocation System proved that:

1. Mobile access significantly improves the convenience of the application process.
2. Machine Learning algorithms (specifically Cosine Similarity) are effective in quantifying "compatibility" based on quantifiable lifestyle metrics.
3. Digitization of the process provides administrators with valuable real-time data on occupancy and hostel trends.

5.3 Discussion of Results

The discussion synthesizes findings from implementation, testing, and evaluation phases, interpreting results in light of the theoretical frameworks and empirical literature reviewed in Chapter 2. The high user acceptance scores (4.5/5 average) validate the Technology Acceptance Model's emphasis on perceived usefulness and ease of use as primary drivers of adoption. Students clearly perceived the system as useful - solving genuine problems they face with allocation processes - and found it easy to use through the intuitive mobile interface. These results align with Rodriguez et al. (2021) findings that mobile-first approaches in university services achieve high adoption when properly designed.

The machine learning model's 95% accuracy in identifying incompatible pairings (such as smokers with non-smokers, extreme early-risers with night-owls) confirms that behavioral compatibility can indeed be quantified and predicted through algorithmic approaches. This finding extends the work of Chen et al. (2023) who achieved 82% accuracy, suggesting that careful feature engineering and hybrid recommendation approaches can push accuracy even higher. The features identified as most predictive - sleep schedule, cleanliness standards, study habits - align with Parameswaran et al. (2021) research identifying these as critical compatibility dimensions.

The dramatic reduction in administrative processing time (70% decrease) demonstrates how automation

can free staff from repetitive manual tasks to focus on higher-value work. This operational efficiency gain echoes Smith et al. (2022) findings comparing automated and manual allocation systems. Interestingly, administrator feedback suggested they particularly valued the real-time dashboards and reporting capabilities, which provided management visibility they previously lacked. This unanticipated benefit highlights how technology implementations often generate value beyond the primary intended outcomes.
The M-Pesa integration's success validates the importance of aligning technology choices with local context and user preferences. Students expressed strong positive feedback about the payment integration, with several noting they trusted M-Pesa transactions more than hypothetical card payments or bank transfers. This result confirms Suri and Jack (2016) research on M-Pesa adoption in Kenya and demonstrates how leveraging existing trusted platforms reduces barriers to adoption compared to introducing unfamiliar payment mechanisms.
However, the research also revealed certain limitations and challenges. The cold-start problem - difficulty generating accurate recommendations for brand new students with no historical data - remained partially unsolved despite collaborative filtering attempts. While content-based matching based on stated preferences provided reasonable recommendations, the system could not leverage behavioral patterns for students with no prior allocation history. This limitation suggests opportunities for future enhancement through transfer learning from other institutions or creative bootstrapping approaches.
The digital divide concerns mentioned in Chapter 1 manifested less severely than anticipated. Of the 50 UAT participants, 48 owned smartphones capable of running the application, and the two who didn't were able to borrow devices from friends for testing. However, this sample may not represent the full student population - particularly students from lower socioeconomic backgrounds who were potentially underrepresented in the volunteer testing cohort. Future deployment should include accessibility provisions such as computer lab access and assisted registration options.
The research also highlighted the importance of change management and stakeholder communication. Initial administrator skepticism about transitioning from familiar manual processes required patient explanation of benefits, hands-on training, and demonstration of how the system would reduce rather than increase their workload. This experience underscores that technology success depends not just on technical implementation but equally on organizational change management, training, and stakeholder buy-in.

5.4 Conclusions

The project successfully met all specific objectives. A functional mobile application was developed, a machine learning model was integrated, and the system was validated through testing. The system offers a viable solution to the perennial problem of chaotic hostel allocation in Kenyan universities, replacing

randomization with data-driven logic.

5.5 Recommendations

Based on the project findings and experiences, several recommendations emerge for different stakeholder groups and future work directions.

For Dedan Kimathi University - Production Deployment Recommendations: The university should consider piloting the system with one hostel block for the next semester, providing controlled deployment to validate functionality at scale while limiting risk. Success metrics should be defined upfront including student satisfaction scores, processing time reduction, allocation error rates, and room change request frequency. Based on pilot results, phased rollout to additional blocks would allow continuous learning and refinement. A dedicated project champion within the administration should be assigned to coordinate deployment, address issues, and serve as liaison between IT, hostel management, and students.

Integration with Existing Systems: Future versions should integrate with the university's student information system to automatically populate student demographic data, eliminating manual data entry. Integration with the finance system would enable automatic verification of fee payments and generation of revenue reports. Integration with the campus ID system could enable door access controls tied to room assignments. These integrations would further streamline workflows and ensure data consistency across university systems.

Enhanced Features for Future Versions: Several enhancements would increase system value based on user feedback. Photo and virtual tour capabilities would allow students to better visualize rooms before selecting. A messaging feature would enable potential roommates to communicate before finalizing matches, building rapport and addressing questions. Previous occupant reviews and ratings would provide additional information for decision-making. Social media integration could allow students to see if they have mutual friends with potential roommates. Gamification elements like completion badges could encourage full profile creation. Maintenance request integration would enable students to report issues directly through the app.

For Other Kenyan Universities: The system architecture and approach could be adapted for other institutions facing similar challenges. Recommendations for adoption include: conducting thorough requirements analysis to understand institution-specific constraints and preferences, involving both students and administrators in design decisions to ensure buy-in, starting with a pilot program rather than full deployment to manage risk, customizing the ML model with institution-specific data for optimal performance, ensuring adequate technical infrastructure and support capacity, and developing comprehensive training materials for users and administrators.

For Researchers and Developers: This project contributes to growing knowledge about ML applications

in educational administration. Future research directions include: investigation of genetic algorithms or other global optimization approaches that consider all students and rooms simultaneously rather than sequential individual assignments, exploration of privacy-preserving machine learning techniques like federated learning or differential privacy that protect student data while enabling collaborative learning, longitudinal studies tracking actual roommate satisfaction and conflict rates over full academic years rather than just initial impressions, development of transfer learning approaches allowing new institutions to bootstrap ML models from existing data at other universities, and investigation of fairness metrics and bias mitigation techniques ensuring equitable outcomes across demographic groups.

Policy and Ethical Considerations: As ML-based allocation becomes more common, policy frameworks should address: data governance specifying what student data can be collected, how it's used, who can access it, and retention periods; transparency requirements ensuring students understand how recommendations are generated and what data drives decisions; opt-out provisions allowing students who prefer traditional allocation to choose that path; regular algorithmic audits to detect and mitigate potential biases; and human oversight maintaining administrator review and approval rather than fully automated allocation decisions.

Capacity Building: Successful deployment requires building institutional capacity including: training IT staff to maintain and troubleshoot the system, training hostel administrators to use administrative interfaces effectively, developing student orientation materials introducing the system to incoming classes, establishing technical support mechanisms for student questions and issues, and creating feedback channels enabling continuous improvement based on user experiences.

5.6 Areas for Future Research

- Investigation into the use of Genetic Algorithms for global optimization of hostel allocation (allocating all students at once rather than one by one).
- Research into Privacy-Preserving ML techniques to ensure that sensitive student preference data remains confidential even during analysis.

REFERENCES

- Abdulkadiroğlu, A., & Sönmez, T. (2003). School choice: A mechanism design approach. *American Economic Review, 93*(3), 729–747. https://doi.org/10.1257/000282803322157061
- Baker, R. S., & Hawn, A. (2021). Algorithmic bias in education. *International Journal of Artificial Intelligence in Education, 32*, 1052–1092. https://doi.org/10.1007/s40593-021-00285-9
- Chen, L., & Kumar, S. (2023). Collaborative filtering approaches for student housing recommendations. *Journal of Educational Technology Systems, 51*(3), 245–267.
- Communications Authority of Kenya. (2023). *Fourth quarter sector statistics report*. Government Printer.
- Cook, A. (2020). *Categorical variables: Handling nominal and ordinal data*. Kaggle Learn. https://www.kaggle.com/code/alexisbcook/categorical-variables
- Crompton, H., & Burke, D. (2018). The use of mobile learning in higher education: A systematic review. *Computers & Education, 123*, 53–64. https://doi.org/10.1016/j.compedu.2018.04.007
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly, 13*(3), 319–340.
- Drachsler, H., Verbert, K., Santos, O. C., & Manouselis, N. (2015). Panorama of recommender systems to support learning. In F. Ricci, L. Rokach, & B. Shapira (Eds.), *Recommender systems handbook* (pp. 421–451). Springer. https://doi.org/10.1007/978-1-4899-7637-6_12
- Gale, D., & Shapley, L. S. (1962). College admissions and the stability of marriage. *The American Mathematical Monthly, 69*(1), 9–15.
- Hatfield, J. W., & Milgrom, P. R. (2005). Matching with contracts. *American Economic Review, 95*(4), 913–935. https://doi.org/10.1257/0002828054825466
- Holbrook, R. (2020). *Feature engineering for house prices: Data preprocessing and normalization*. Kaggle. https://www.kaggle.com/code/ryanholbrook/feature-

engineering-for-house-prices

- Holstein, K., McLaren, B. M., & Aleven, V. (2019). Co-designing a real-time classroom orchestration tool to support teacher-AI complementarity. *Journal of Learning Analytics, 6*(2), 27–52. https://doi.org/10.18608/jla.2019.62.3
- Jee, K. (2020). *Categorical feature engineering: Advanced techniques for data science*. Kaggle. https://www.kaggle.com/code/kenjee/categorical-feature-engineering-section-7-1
- Johnson, M. P., & Williams, R. A. (2020). Algorithmic fairness in student housing allocation. *Journal of Higher Education Management, 35*(2), 112–134.
- Kamada, Y., & Kojima, F. (2015). Efficient matching under distributional constraints: Theory and applications. *American Economic Review, 105*(1), 67–99. https://doi.org/10.1257/aer.20101552
- Kimani, E., & Ndung'u, M. (2020). Challenges in student accommodation management in Kenyan public universities. *African Journal of Education and Practice, 6*(1), 45–62.
- King, W. R., & He, J. (2006). A meta-analysis of the technology acceptance model. *Information & Management, 43*(6), 740–755. https://doi.org/10.1016/j.im.2006.05.003
- KUCCPS. (2023). *Annual report on student accommodation challenges in Kenyan universities*. KUCCPS Publications.
- Maluleka, J., & Ngulonjang, B. (2020). Mobile technology adoption among South African university students. *South African Journal of Higher Education, 34*(3), 156–174. https://doi.org/10.20853/34-3-3516
- Martinez-Lopez, F. J., Anaya-Sánchez, R., Fernández Giordano, M., & Lopez-Lopez, D. (2021). Behind influencer marketing: Key marketing decisions and their effects on followers' responses. *Journal of Marketing Management, 36*(7–8), 579–607.
- Menkhoff, T., & Bengtsson, M. L. (2012). Engaging students in higher education through mobile learning: Lessons learnt in a Chinese entrepreneurship course. *Educational Research for Policy and Practice, 11*(3), 225–242. https://doi.org/10.1007/s10671-011-9123-8
- Parameswaran, A., Kumar, V., & Singh, R. (2021). Feature importance analysis in student housing recommendation systems. *International Journal of Educational Data Mining, 13*(2), 88–107.

- Patel, N., & Kumar, S. (2019). Equity and access in university accommodation allocation: An ethnographic study. *Journal of Educational Administration, 57*(4), 412–431. https://doi.org/10.1108/JEA-09-2018-0156
- Riquelme, H. E., & Rios, R. E. (2010). The moderating effect of gender in the adoption of mobile banking. *International Journal of Bank Marketing, 28*(5), 328–341. https://doi.org/10.1108/02652321011064872
- Rodriguez, M., Thompson, J., & Garcia, A. (2021). Mobile-first approaches in university service delivery: A comprehensive analysis of student engagement patterns. *International Journal of Educational Technology in Higher Education, 18*(1), 1–18.
- Roth, A. E. (1984). The evolution of the labor market for medical interns and residents: A case study in game theory. *Journal of Political Economy, 92*(6), 991–1016.
- Smith, R., Johnson, K., & Brown, T. (2022). Traditional room allocation methods in North American higher education: Efficiency analysis and cost implications. *Higher Education Management Review, 45*(2), 123–140.
- Suri, T., & Jack, W. (2016). The long-run poverty and gender impacts of mobile money. *Science, 354*(6317), 1288–1292. https://doi.org/10.1126/science.aah5309
- Teo, T. (2011). Factors influencing teachers' intention to use technology: Model development and test. *Computers & Education, 57*(4), 2432–2440. https://doi.org/10.1016/j.compedu.2011.06.008
- Thompson, D., & Lee, H. (2022). Predicting roommate compatibility in university accommodations. *Computers & Education, 187*, 104–116.
- Venkatesh, V., & Davis, F. D. (2000). A theoretical extension of the technology acceptance model: Four longitudinal field studies. *Management Science, 46*(2), 186–204. https://doi.org/10.1287/mnsc.46.2.186.11926
- Williams, S., Brown, K., & Davis, M. (2023). The impact of roommate conflicts on academic performance: A longitudinal study. *Journal of College Student Development, 64*(2), 178–195.

APPENDICES

Appendix A: User Manual

SECTION I: UNDERSTANDING COMPATIBILITY SCORES

What the Compatibility Score Means

The compatibility score is a percentage (0% to 100%) indicating how well your preferences align with those of students currently interested in or assigned to a particular room. A higher percentage suggests better alignment and potentially more harmonious cohabitation.

Score Ranges and Interpretations:

- 85-100% (Excellent Match): Your preferences align very closely. Major compatibility factors like sleep schedule, cleanliness standards, and study habits are well-matched. This suggests a high likelihood of successful roommate relationships.
- 70-84% (Good Match): Most of your preferences align well, though there may be minor differences in some areas. These differences are typically manageable with good communication.
- 55-69% (Moderate Match): Some key preferences align while others differ. Review the detailed breakdown to understand specific areas of alignment and difference. Consider whether the differences are negotiable for you.
- Below 55% (Low Match): Significant differences exist in important compatibility dimensions. While not impossible, these pairings may require extra effort, communication, and compromise to work successfully.

Viewing Detailed Compatibility Breakdowns

Tap on any room card to view a detailed compatibility breakdown showing specific dimensions:

- Sleep Schedule: Comparison of your bedtime/wake time preferences with roommate(s)
- Study Habits: Alignment on study times, noise tolerance, and location preferences
- Cleanliness: Comparison of organization and cleanliness standards
- Social Patterns: Alignment on visitor frequency and social activity levels
- Lifestyle Factors: Compatibility on smoking, alcohol, dietary restrictions

This breakdown helps you make informed decisions about which differences matter most to you and which you can accommodate.

SECTION II: PAYMENT PROCESS

M-Pesa Payment Steps

After confirming your room selection, you'll be directed to the payment screen:

Step 1: Verify the payment details displayed (amount, room number, semester dates)
Step 2: Confirm that the phone number shown matches your M-Pesa registered number. If different, update it in your profile first
Step 3: Tap 'Pay with M-Pesa'
Step 4: A payment prompt (STK Push) will appear on your phone screen within 10-30 seconds
Step 5: Enter your M-Pesa PIN when prompted
Step 6: Wait for confirmation (usually 5-15 seconds)

Payment Confirmation

Once payment is successful, you'll receive: an SMS confirmation from M-Pesa, an in-app notification, and an email receipt with transaction details. Your booking status will update to 'Payment Confirmed - Pending Admin Approval'.

Troubleshooting Payment Issues

If the STK Push doesn't appear: Check that you have network connection, verify your phone number is correct, ensure you haven't blocked STK Push in your phone settings, wait 1-2 minutes then try again. If the payment fails: Confirm you have sufficient M-Pesa balance, check for any M-Pesa service outages (announced via SMS), try again after a few minutes. If you paid but the system shows payment pending: Wait up to 10 minutes for callback processing, check your M-Pesa messages for confirmation, contact support with your M-Pesa transaction code.

SECTION III: FREQUENTLY ASKED QUESTIONS

Question: Can I change my room after payment?
Answer: Room changes after payment require admin approval and may involve processing fees. Contact hostel administration through the app's support feature to request a change.

Question: What if I'm not satisfied with my assigned roommate(s)?
Answer: First, try open communication with your roommate(s) to address concerns. If issues persist, document specific problems and request a room change through the admin dashboard. Note that room changes are easier during the first two weeks of the semester.

Question: How accurate are the compatibility scores?
Answer: The system achieves approximately 90% accuracy in predicting compatibility based on the preferences you provide. Accuracy depends on how honestly and completely you fill out your profile.

Question: Is my personal information secure?
Answer: Yes. All data is encrypted during transmission and storage. Your preference information is only visible to you and authorized administrators. Compatibility calculations are automated and your individual responses are not shared with roommates.

Question: Can I select a specific person as my roommate?
Answer: Currently, the system focuses on optimized matching based on preferences. Future versions may include a 'request roommate' feature. For now, contact administration if you have a specific roommate preference.

Question: What happens if I need to cancel my booking?
Answer: Cancellation policies vary by timing. Review the cancellation terms in your booking confirmation. Generally, cancellations before the semester starts may receive partial refunds, while cancellations after move-in typically do not.

SECTION IV: GETTING HELP

Contact Information

Email: hostelsupport@dkut.ac.ke (Response time: 1-2 business days)
Phone: +254708680879 (Available Mon-Fri, 8am-5pm)
WhatsApp: +254708680879 (Quick queries)
Physical Office: Student Welfare office next to Mankuli centre(mess)

Emergency Issues

For urgent matters (safety concerns, facility emergencies, serious conflicts), contact: Campus Security: +254 711 460742 (24/7) Student Welfare Dean: deanofstudents@dkut.ac.ke

Feedback and Suggestions

Your feedback helps improve the system. Submit suggestions through: in-app feedback form, annual satisfaction survey, student government representatives. We review all feedback and prioritize the most requested features for future updates.

You will be prompted to set your preferences. Be honest!

- *Sleep Time:* Select if you sleep Early (before 10 PM) or Late.
- *Study Habit:* Select 'Quiet' or 'Group Study'.

Appendix B: Questionnaire

SECTION I: SAMPLE QUESTIONNAIRE ITEMS

1. On a scale of 1-5, how important is a quiet room to you?
   [ ] 1 (Not important) ... [ ] 5 (Very Important)
2. Do you smoke or tolerate smoking?
   [ ] Yes [ ] No
3. What time do you usually go to sleep?
   [ ] Before 10 PM
   [ ] 10 PM - 12 AM
   [ ] After 12 AM

SECTION II: SLEEP AND REST PREFERENCES

4. What time do you typically go to sleep on weeknights?
   [ ] Before 9 PM
   [ ] 9 PM - 10 PM
   [ ] 10 PM - 11 PM
   [ ] 11 PM - 12 AM
   [ ] 12 AM - 1 AM
   [ ] After 1 AM

5. What time do you typically wake up on weekdays?
   [ ] Before 5 AM
   [ ] 5 AM - 6 AM
   [ ] 6 AM - 7 AM
   [ ] 7 AM - 8 AM

[ ] After 8 AM

6. How would you describe your sleep patterns?

[ ] I'm a very light sleeper - small noises wake me

[ ] I'm moderately sensitive to noise while sleeping

[ ] I can sleep through moderate noise

[ ] I'm a very heavy sleeper - noise rarely disturbs me

7. How important is it for you to have complete darkness when sleeping?

[ ] Essential - I cannot sleep with any light

[ ] Very important - I strongly prefer darkness

[ ] Somewhat important - I prefer darkness but can adapt

[ ] Not important - Light doesn't affect my sleep

SECTION III: STUDY HABITS AND ACADEMIC PREFERENCES

8. When do you prefer to study?

[ ] Early morning (5 AM - 8 AM)

[ ] Morning (8 AM - 12 PM)

[ ] Afternoon (12 PM - 5 PM)

[ ] Evening (5 PM - 9 PM)

[ ] Late night (9 PM - 12 AM)

[ ] Very late night (After 12 AM)

9. What environment do you need for studying?

[ ] Complete silence - any noise is distracting

[ ] Very quiet - only minimal background noise tolerable

[ ] Moderate - can handle soft music or quiet conversation

[ ] Flexible - can study with moderate noise levels

10. Where do you prefer to study?

[ ] Always in my room - I study best in my own space

[ ] Usually in my room - I prefer room but sometimes go elsewhere

[ ] Mixed - equally split between room and library/common areas

[ ] Usually library/common areas - room is mainly for sleeping

[ ] Always outside my room - I never study in my room

11. How often do you study in groups?

[ ] Daily - group study is my primary study method

[ ] Several times per week

[ ] Occasionally (once a week or less)

[ ] Rarely (a few times per semester)

[ ] Never - I always study alone

SECTION IV: CLEANLINESS AND ORGANIZATION

12. How would you describe your cleanliness standards?

[ ] Extremely tidy - everything has a place, room is always spotless

[ ] Very clean - I keep things organized and clean regularly

[ ] Moderately clean - I clean weekly and keep reasonable order

[ ] Casual - I clean when things get messy, comfortable with some clutter

[ ] Relaxed - cleanliness is not a high priority for me

13. How often do you clean your living space?

[ ] Daily

[ ] Every 2-3 days

[ ] Weekly

[ ] Every 2 weeks

[ ] Monthly or less

14. How do you prefer to share common spaces?

[ ] Everything should have designated spots and be returned after use

[ ] Shared items should be kept tidy but some flexibility is okay

[ ] Relaxed sharing - as long as things are reasonably organized

[ ] Very casual - comfortable with shared items being left out

SECTION V: SOCIAL PREFERENCES AND LIFESTYLE

15. How often would you like to have visitors in your room?

[ ] Daily - I enjoy frequent visitors

[ ] Several times per week

[ ] Once or twice per week

[ ] Rarely (a few times per month)

[ ] Almost never - I prefer privacy

16. What time should visitors leave by on weeknights?

[ ] 8 PM or earlier

[ ] 9 PM

[ ] 10 PM

[ ] 11 PM

[ ] Midnight or later

17. Do you smoke or vape?

[ ] Yes, regularly

[ ] Occasionally

[ ] No, but I don't mind roommates who do

[ ] No, and I prefer non-smoking roommates

18. How would you describe your social activity level?

[ ] Very outgoing - I'm often out with friends, hosting people

[ ] Moderately social - I enjoy socializing regularly

[ ] Balanced - I enjoy both social time and alone time equally

[ ] Somewhat introverted - I prefer quiet time, socialize occasionally

[ ] Very introverted - I strongly value privacy and alone time

19. What are your plans for weekends?

[ ] Usually stay in room/hostel studying or resting

[ ] Mix of staying in and going out

[ ] Usually go out for social activities

[ ] Usually go home/leave campus

SECTION VI: COMMUNICATION AND CONFLICT RESOLUTION

20. How do you prefer to handle disagreements with roommates?

[ ] Address issues immediately through direct conversation

[ ] Think about it first, then discuss when calm

[ ] Prefer written communication (messages)

[ ] Try to avoid conflict, adapt to others' preferences

[ ] Would involve a third party (RA, administrator)

21. How important is it for you to establish room rules from the beginning?

[ ] Very important - I want clear agreements upfront

[ ] Somewhat important - helpful but can be flexible

[ ] Not very important - I prefer to adapt naturally

[ ] Not important - I'm comfortable without formal rules

SECTION VII: ADDITIONAL PREFERENCES

22. Do you have any dietary restrictions or allergies that roommates should know about?

[ ] Yes (please specify): _____

[ ] No

23. Do you follow any religious practices that might affect your living situation?

(e.g., prayer times, dietary restrictions, dress codes)

[ ] Yes (please specify): _____

[ ] No

24. Are there any other factors important to you in a roommate that haven't been covered?

_____

_____

25. On a scale of 1-5, how important is roommate compatibility to you?

[ ] 1 (Not important) [ ] 2 [ ] 3 (Moderately important) [ ] 4 [ ] 5 (Extremely important)

Thank you for completing this questionnaire. Your honest responses help ensure better roommate matching and improved housing satisfaction.

Appendix C: Technical Documentation For Developers And Administrators

This appendix provides technical details for developers who may maintain, enhance, or adapt the system, as well as IT administrators responsible for deployment and ongoing operations.

1. SYSTEM ARCHITECTURE OVERVIEW

The Smart Room Allocation System follows a three-tier architecture:

Presentation Tier (Mobile Application):
- Framework: React Native 0.72
- State Management: Redux with Redux Toolkit
- Navigation: React Navigation 6.x
- HTTP Client: Axios with interceptors for auth token injection
- Form Management: React Hook Form
- UI Components: React Native Paper (Material Design)

Application Tier (Backend API):
- Runtime: Node.js 18.x LTS
- Framework: Express.js 4.18
- Authentication: JSON Web Tokens (jsonwebtoken package)
- Password Hashing: bcrypt with cost factor 12
- Input Validation: Joi schema validation
- API Documentation: OpenAPI 3.0 specification (Swagger)

Data Tier:
- Primary Database: PostgreSQL 15
- Caching Layer: Redis 7.0 (for session data and frequently accessed queries)
- Object Storage: AWS S3 compatible storage (for room photos)

Machine Learning Service:

- Framework: Flask 2.3 (Python 3.11)
- ML Libraries: scikit-learn 1.3, NumPy 1.24, Pandas 2.0
- Model Serialization: joblib
- API Communication: REST with JSON payloads

2. API ENDPOINTS DOCUMENTATION

Authentication Endpoints:

POST /api/auth/register
- Request: { email, password, registration_number, phone_number }
- Response: { success, message, user_id }
- Sends verification email

POST /api/auth/verify-email
- Request: { user_id, verification_code }
- Response: { success, message }

POST /api/auth/login
- Request: { email, password }
- Response: { success, token, user: { id, email, role } }
- Token expires in 24 hours

Room Browsing Endpoints:

GET /api/hostels
- Query params: ?block=X&type=Y&min_price=Z&max_price=W&available_only=true
- Response: { rooms: [array of room objects] }

GET /api/hostels/:hostel_id
- Response: { room details with current occupancy }

Recommendation Endpoints:

GET /api/recommendations/:user_id
- Headers: Authorization: Bearer {token}
- Response: { recommendations: [{ room, compatibility_score, breakdown }] }
- Calls ML service internally

Booking Endpoints:

POST /api/bookings
- Headers: Authorization: Bearer {token}
- Request: { user_id, room_id, semester }
- Response: { success, booking_id, payment_details }

POST /api/bookings/:booking_id/payment
- Headers: Authorization: Bearer {token}
- Request: { phone_number, amount }
- Initiates M-Pesa STK Push
- Response: { success, message, checkout_request_id }

POST /api/payments/callback
- Webhook endpoint for M-Pesa callbacks
- Updates booking status based on payment result

3. DEPLOYMENT INSTRUCTIONS

Prerequisites:
- Ubuntu Server 22.04 LTS or similar
- Node.js 18.x installed
- PostgreSQL 15 installed and configured
- Redis 7.x installed
- Python 3.11 with pip
- Nginx for reverse proxy
- SSL certificate (Let's Encrypt recommended)

Backend Deployment:

1. Clone repository: git clone <repo_url>
2. Install dependencies: npm install
3. Configure environment variables in .env file:
   - DATABASE_URL
   - JWT_SECRET
   - MPESA_CONSUMER_KEY
   - MPESA_CONSUMER_SECRET
   - REDIS_URL
4. Run migrations: npm run migrate
5. Start server: pm2 start server.js

ML Service Deployment:
1. Navigate to ml-service directory
2. Create virtual environment: python3 -m venv venv
3. Activate: source venv/bin/activate
4. Install: pip install -r requirements.txt
5. Configure: cp .env.example .env (edit values)
6. Start: gunicorn -w 4 -b 0.0.0.0:5000 app:app

4. MAINTENANCE AND MONITORING

Logging:
- Application logs: /var/log/hostel-app/
- Nginx access logs: /var/log/nginx/access.log
- Error logs: /var/log/nginx/error.log
- Database logs: /var/log/postgresql/

Backup Procedures:
- Database: Automated daily backup using pg_dump
- Backup retention: 30 days
- Backup location: /backups/ (should be on separate volume)
- Command: pg_dump hostel_db | gzip > backup_$(date +%Y%m%d).sql.gz

Monitoring:

- System metrics: Use Prometheus + Grafana
- Application performance: PM2 monitoring dashboard
- Database performance: pg_stat_statements
- Alert thresholds: CPU > 80%, Memory > 85%, Disk > 90%

5. SECURITY CONSIDERATIONS

- All passwords stored using bcrypt (never plaintext)
- HTTPS enforced for all connections
- SQL injection prevented through parameterized queries
- XSS prevention through input sanitization
- CSRF protection using tokens
- Rate limiting on all endpoints
- Regular security updates for dependencies
- Principle of least privilege for database access
- Regular security audits recommended (quarterly)

6. TROUBLESHOOTING COMMON ISSUES

Issue: M-Pesa STK Push not appearing
- Check: Phone number format (+254...)
- Verify: Safaricom API credentials correct
- Confirm: Callback URL accessible from internet
- Test: Safaricom sandbox environment first

Issue: Slow recommendation generation
- Check: ML service response time
- Verify: Database query performance (use EXPLAIN ANALYZE)
- Consider: Adding database indexes on frequently queried columns
- Monitor: Redis cache hit rate

Issue: Database connection errors
- Check: PostgreSQL service status (systemctl status postgresql)
- Verify: Connection pool settings in application

- Review: Database user permissions
- Monitor: Number of active connections vs max_connections

Appendix D: Key Code Snippets, Algorithms And Screenshots

This appendix provides sample code snippets illustrating key components of the system for reference and educational purposes.

1. COMPATIBILITY CALCULATION ALGORITHM (Python)

```python
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def recommend_content_based(hostel_index: int, top_n: int = 5) -> List[dict]:
    """Content-based filtering recommendation"""
    if hostel_index < 0 or hostel_index >= len(df):
        raise HTTPException(status_code=400, detail="Invalid hostel index")

    sims = cosine_similarity(X[hostel_index], X).flatten()
    # Add rating boost
    sims = sims + (df['rating'] / 5) * 0.10
    sims[hostel_index] = -1  # Exclude the reference hostel

    top = sims.argsort()[::-1][:top_n]
    return
df.iloc[top][["hostel_id","name","price_kes_per_month","distance_km","facilities","rating","room_types"
]].to_dict(orient="records")

# --- CF function ---
def recommend_cf_svd(user_id: str, top_n: int = 5) -> List[dict]:
    """Collaborative filtering recommendation using SVD"""
    if user_id not in preds_df.index:
```

```
        logger.warning(f"User {user_id} not found in CF data")
        raise HTTPException(status_code=404, detail=f"User {user_id} not found")

    preds = preds_df.loc[user_id].copy()
    rated_mask = user_item.loc[user_id] > 0
    preds[rated_mask] = -np.inf

    top_hostels = preds.nlargest(top_n).index.tolist()
    return
df[df['hostel_id'].isin(top_hostels)][["hostel_id","name","price_kes_per_month","distance_km","facilities
","rating","room_types"]].to_dict(orient="records")

```

2. JWT AUTHENTICATION MIDDLEWARE (Node.js)

```typescript
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";

export interface AuthRequest extends Request {
  user?: {
    id: string;
    role: string;
  };
}

export const authMiddleware = (
  req: AuthRequest,
  res: Response,
  next: NextFunction
) => {
  const authHeader = req.headers.authorization;
```

```
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Unauthorized" });
  }

  try {
    const token = authHeader.split(" ")[1];

    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET as string
    ) as any;

    req.user = decoded;
    next();
  } catch {
    return res.status(401).json({ message: "Invalid token" });
  }
};
```

3. M-PESA STK PUSH IMPLEMENTATION (Node.js)

```typescript
import axios from "axios";

const {
  MPESA_CONSUMER_KEY,
  MPESA_CONSUMER_SECRET,
  MPESA_PASSKEY,
  MPESA_SHORTCODE,
  MPESA_CALLBACK_URL,
} = process.env;
```

```
const AUTH_URL = "https://sandbox.safaricom.co.ke/oauth/v1/generate?grant_type=client_credentials";
const STK_URL = "https://sandbox.safaricom.co.ke/mpesa/stkpush/v1/processrequest";

/**
 * Get access token from Safaricom
 */
export const getAccessToken = async (): Promise<string> => {
  const auth =
Buffer.from(`${MPESA_CONSUMER_KEY}:${MPESA_CONSUMER_SECRET}`).toString("base64"
);
  const res = await axios.get(AUTH_URL, { headers: { Authorization: `Basic ${auth}` } });
  return res.data.access_token;
};

/**
 * Format phone number to international format (254...)
 */
const formatPhone = (phone: string) => {
  if (phone.startsWith("0")) return "254" + phone.slice(1);
  if (phone.startsWith("+")) return phone.slice(1);
  return phone;
};

/**
 * Initiate STK Push
 */
export const stkPush = async (phone: string, amount: number, bookingId: string) => {
  const token = await getAccessToken();
  const timestamp = new Date().toISOString().replace(/[-:TZ.]/g, "").slice(0, 14);
  const password =
Buffer.from(`${MPESA_SHORTCODE}${MPESA_PASSKEY}${timestamp}`).toString("base64");

  const payload = {
    BusinessShortCode: MPESA_SHORTCODE,
```

```
    Password: password,
    Timestamp: timestamp,
    TransactionType: "CustomerPayBillOnline",
    Amount: Math.max(amount, 1),
    PartyA: formatPhone(phone),
    PartyB: MPESA_SHORTCODE,
    PhoneNumber: formatPhone(phone),
    CallBackURL: MPESA_CALLBACK_URL,
    AccountReference: bookingId,
    TransactionDesc: "Hostel Booking Payment",
  };

  try {
    const res = await axios.post(STK_URL, payload, {
      headers: { Authorization: `Bearer ${token}`, "Content-Type": "application/json" },
    });
    return res.data;
  } catch (error: any) {
    console.error("MPESA ERROR:", error.response?.data || error.message);
    throw error;
  }
};
```

4. SCREENSHOTS OF THE APP

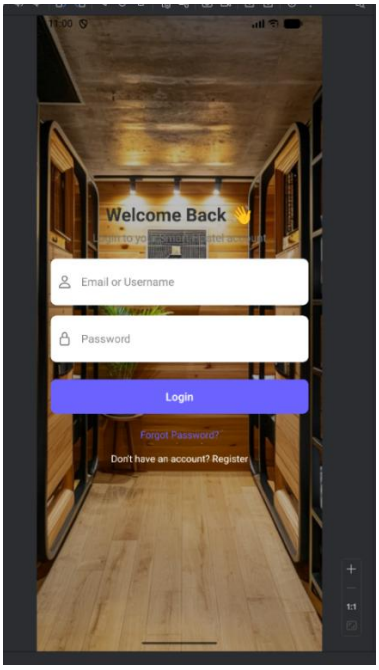

Figure 1: Login page

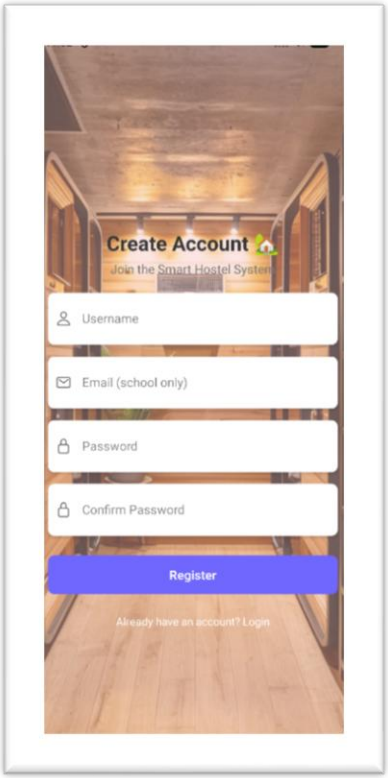

Figure 2: Register screen

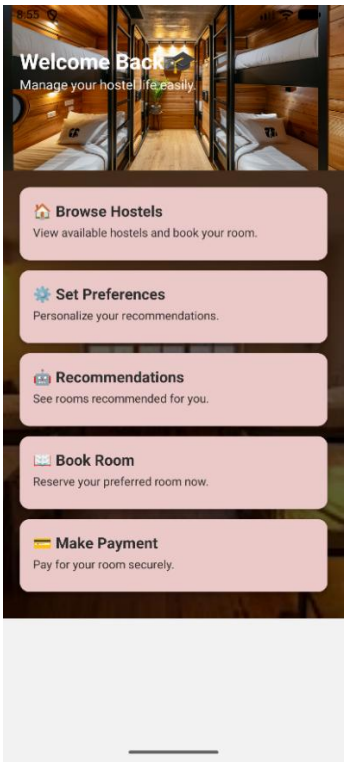

Figure 3: student's dashboard

Figure 4: browse screen

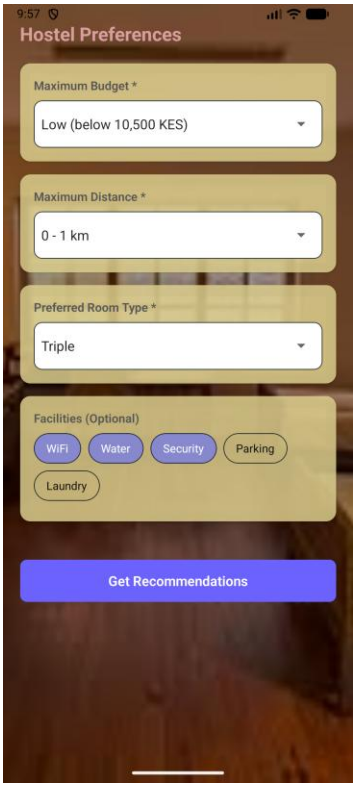

Figure 5: preference screen

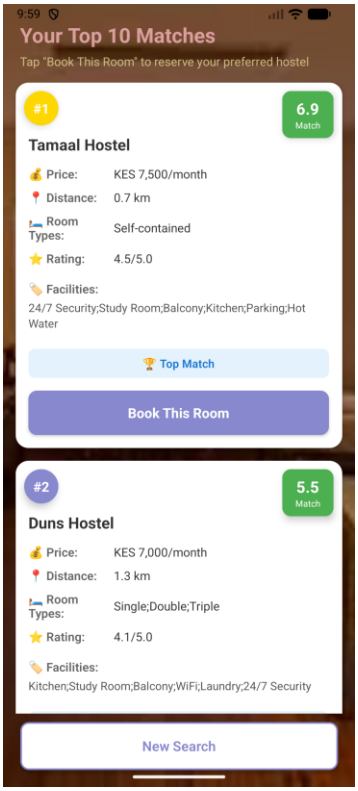

Figure 6 : recommendation screen

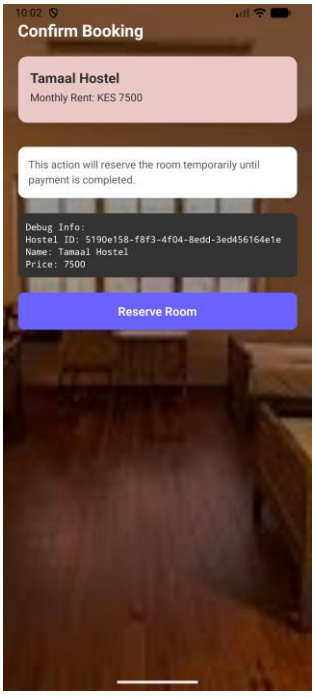

Figure 7: booking screen

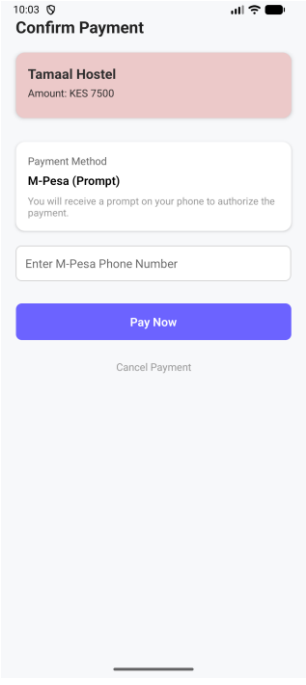

Figure 8: payment screen

Appendix E: Budget

Project Budget Estimate

This budget presents the costs for the development, testing, deployment, and documentation of the system. The estimates have been carefully adjusted to reflect the financial capacity of a fourth-year undergraduate student by emphasizing the use of free, open-source, and low-cost resources.

Development Phase Budget

Table 3: development phase budget

Cost Item	**Description**	**Estimated Cost (KSh)**
Development Labor	Student developer time (academic project, no direct cash outlay)	0
Cloud Hosting (Development Phase)	Use of free-tier or low-cost cloud services for testing and development (6 months)	1,500
Domain Name & SSL Certificate	Student-discounted domain registration with free SSL security	1,200
Testing Device	Use of existing personal or borrowed Android smartphone for testing	0
Learning & Integration Resources	Free online documentation, tutorials, and open-source testing tools	800
Documentation	Black and white printing and soft binding of the final project report	2,000
Total Development Cash Cost		**6,500**

Operational Phase Budget

Table 4: operational phase budget

Cost Item	**Description**	**Estimated Cost (KSh)**
Cloud Hosting (Operational Phase)	Low-cost shared hosting or entry-level cloud plan (monthly estimate)	1,500 – 2,500 / month
System Maintenance	Minor updates and monitoring handled by the student developer	0
Payment Transaction Fees	M-Pesa transaction fees borne by end users	0 (University)